

Assignment 4 extra

Hadeer Ghoneim

Verification Diploma

Q1

code_cvr_enc - Notepad

File Edit Format View Help

Coverage Report by instance with details

```
=====
=== Instance: /\priority_enc_tb#DUT_enc
=== Design Unit: work.priority_enc
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	7	7	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /\priority_enc_tb#DUT_enc

Line	Item	Count	Source
----	----	----	-----
File priority_enc.v			
-----IF Branch-----			
10		27	Count coming in to IF
10	1	2	if (rst)
12	1	25	else

Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----			
13		25	Count coming in to CASE
14	1	3	4'b1000: Y <= 2'b00;
15	1	6	4'bx100: Y <= 2'b01;
16	1	5	4'bXX10: Y <= 2'b10;
17	1	9	4'bXXX1: Y <= 2'b11;
18	1	2	default: Y <= 2'bxx;

Branch totals: 5 hits of 5 branches = 100.00%

Coverage Report by instance with details

```
=====
=== Instance: /\ALU_tb#DUT_ALU
=== Design Unit: work.ALU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	6	6	0	100.00%

=====Branch Details=====

Branch Coverage for instance /\ALU_tb#DUT_ALU

Line	Item	Count	Source
----	----	-----	-----
File ALU.v			
-----CASE Branch-----			
22		42	Count coming in to CASE
23	1	16	Add: Alu_out = A + B;
24	1	16	Sub: Alu_out = A - B;
25	1	5	Not_A: Alu_out = ~A;
26	1	5	ReductionOR_B: Alu_out = B;

Branch totals: 4 hits of 4 branches = 100.00%

Line	Item	Count	Source
----	----	-----	-----
-----IF Branch-----			
37		43	Count coming in to IF
37	1	4	if (reset)
39	1	39	else

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\priority_enc_tb#DUT_enc --

Line	Item	Count	Source
----	----	-----	-----
File priority_enc.v			
1			module priority_enc (
2			input clk,
3			input rst,
4			input [3:0] D,
5			output reg [1:0] Y,
6			output reg valid
7			);
8			
9	1	27	always @(posedge clk) begin
10			if (rst)
11	1	2	Y <= 2'b00;
12			else
13			casex (D)
14	1	3	4'b1000: Y <= 2'b00;
15	1	6	4'bx100: Y <= 2'b01;

code_cvr_alu_fixed - Notepad

File Edit Format View Help

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\ALU_tb#DUT_ALU --

Line	Item	Count	Source
----	----	-----	-----
File ALU.v			
1			module ALU (
2			input clk,
3			input reset,
4			input [1:0] Opcode, // The opcode
5			input signed [3:0] A, // Input data A in 2's complement
6			input signed [3:0] B, // Input data B in 2's complement
7			
8			output reg signed [4:0] C // ALU output in 2's complement
9			
10			);
11			
12			reg signed [4:0] Alu_out; // ALU output in 2's complement
13			
14			localparam Add = 2'b00; // A + B
15			localparam Sub = 2'b01; // A - B

```

15          1          6          4'bX100: Y <= 2'b01;
16          1          5          4'bXX10: Y <= 2'b10;
17          1          9          4'bXXX1: Y <= 2'b11;
18          1          2          default: Y <= 2'bxx;
19
20          endcase
21          1          27         valid <= (~|D)? 1'b0: 1'b1 ;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Toggles	18	18	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\priority_enc_tb#DUT_enc --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
D[0-3]	1	1	100.00
Y[1-0]	1	1	100.00
clk	1	1	100.00
rst	1	1	100.00
valid	1	1	100.00

Total Node Count = 9

Toggled Node Count = 9

Untoggled Node Count = 0

Toggle Coverage = 100.00% (18 of 18 bins)

Total Coverage By Instance (filtered view): 100.00%

```

16                                localparam          Not_A              = 2'b10; // ~A
17                                localparam          ReductionOR_B = 2'b11; // |B
18
19                                // Do the operation
20                                //covered all cases no need to use default
21                                1                    42          always @* begin
22                                    case (Opcode)
23                                1                    16          Add:          Alu_out = A + B;
24                                1                    16          Sub:          Alu_out = A - B;
25                                1                    5          Not_A:          Alu_out = ~A;
26                                1                    5          ReductionOR_B: Alu_out = |B;
27                                    default: Alu_out = 5'b0;
28                                    endcase
29                                end // always @ *
30
31                                // Register output C
32                                // why do not remove c ??
33                                // why use internal signal alu_out!!!?
34                                //maybe to avoid latches???!!
35                                //you want alu with register?
36                                1                    43          always @(posedge clk or posedge reset) begin

```

```

35                                     //you want alu with register?
36             1                      43         always @(posedge clk or posedge reset) begin
37
38             1                      4           C <= 5'b0;
39
39                                     else
40             1                      39          C<= Alu_out;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	44	44	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\ALU_tb#DUT_ALU --

Node	1H->0L	0L->1H	"Coverage"
A[0-3]	1	1	100.00
Alu_out[4-0]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
Opcode[0-1]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 22
 Toggled Node Count = 22
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%


```

1  module priority_enc (
2      input  clk,
3      input  rst,
4      input  [3:0] D,
5      output reg [1:0] Y,
6      output reg valid
7  );
8
9  always @(posedge clk) begin
10     if (rst) begin
11         Y <= 2'b00;
12         valid <= 1'b0;    // Fix 1: valid = 0 at reset
13     end
14     else begin            // Fix 2: casex and valid in begin-end
15         casex (D)
16             4'b1000: Y <= 2'b00;
17             4'bX100: Y <= 2'b01;
18             4'bXX10: Y <= 2'b10;
19             4'bXXX1: Y <= 2'b11;
20         endcase
21
22         valid <= (~|D) ? 1'b0 : 1'b1;
23     end
24 end
25 endmodule
26

```

```
1 module ALU (  
2     input wire clk,  
3     input wire reset,  
4     input wire [1:0] Opcode,      // ALU opcode  
5     input wire signed [3:0] A, // Input A in 2's complement  
6     input wire signed [3:0] B, // Input B in 2's complement  
7     output reg signed [4:0] C    // Output in 2's complement  
8 );  
9  
10    // Internal signal to hold result before register  
11    reg signed [4:0] Alu_out;  
12  
13    // ALU operation codes  
14    localparam Add      = 2'b00; // A + B  
15    localparam Sub      = 2'b01; // A - B  
16    localparam Not_A    = 2'b10; // ~A  
17    localparam ReductionOR_B = 2'b11; // |B  
18  
19    // Combinational ALU logic  
20    always @(*) begin  
21        case (Opcode)  
22            Add:      Alu_out = A + B;      // Addition  
23            Sub:      Alu_out = A - B;      // Subtraction  
24            Not_A:    Alu_out = ~A;         // Bitwise NOT of A  
25            ReductionOR_B: Alu_out = |B;    // Reduction OR of B  
26            default:  Alu_out = 5'd0;      // Should never happen in testing  
27        endcase  
28    end  
29  
30    // Sequential block to register the output  
31    always @(posedge clk or posedge reset) begin  
32        if (reset)  
33            C <= 5'd0;  
34        else  
35            C <= Alu_out;  
36    end
```

```
29    // Sequential block to register the output  
30    always @(posedge clk or posedge reset) begin  
31        if (reset)  
32            C <= 5'd0;  
33        else  
34            C <= Alu_out;  
35    end  
36  
37    endmodule  
38
```

```

1  vlib work
2  vlog ALU.v ALU_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.ALU_tb -cover
4  add wave *
5  coverage save ALU_tb.ucdb -onexit
6  run -all

```

```

1  vlib work
2  vlog priority_enc.v priority_enc_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.priority_enc_tb -cover
4  add wave *
5  coverage save priority_enc_tb.ucdb -onexit
6  run -all

```

```

1  module priority_enc_tb ();
2
3      reg [3:0] D_tb;
4      logic [1:0] Y_tb;
5      reg valid_tb;
6      bit clk;
7      bit rst;
8
9      priority_enc DUT_enc (clk,rst,D_tb,Y_tb,valid_tb);
10
11      always begin
12          #5 clk = ~clk;
13      end
14
15      // COUNTER_1
16
17      task asser_reset ();
18          @(negedge clk);
19          D_tb = 0 ;
20          rst = 1;
21          // === Assertions for reset ===
22          @(posedge clk);
23          assert (Y_tb == 2'b00)
24              else $error("RESET ASSERTION FAILED: Expected Y=00, Got Y=%b at time %0t", Y_tb, $time);
25
26          assert (valid_tb == 0 || valid_tb == 1'bx)
27              else $error("RESET ASSERTION FAILED: Expected valid=0, Got valid=%b at time %0t", valid_tb, $time);
28          rst = 0;
29      endtask
30
31

```

```

33 // COUNTER_2
34 initial begin
35     clk = 0;
36     rst = 1; // Assert reset at time 0
37     D_tb = 4'b0000;
38
39     repeat(2) @(posedge clk);
40     rst = 0; // De-assert reset
41
42     asser_reset();
43
44     // CHECK_1_past_state valid was 0
45     D_tb = 4'b1000;
46     @(posedge clk);
47     assert(Y_tb == 2'b00) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
48     assert(valid_tb == 0) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
49
50     // CHECK_1
51     D_tb = 4'b1000;
52     @(posedge clk);
53     assert(Y_tb == 2'b00) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
54     assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
55
56     // CHECK_2_past_state y was 00
57     D_tb = 4'b0100;
58     @(posedge clk);
59     assert(Y_tb == 2'b00) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
60     assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
61
62     // CHECK_2
63     D_tb = 4'b0100;
64     @(posedge clk);
65     assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
66     assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);

```

```

62 // CHECK_2
63 D_tb = 4'b0100;
64 @(posedge clk);
65 assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
66 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
67
68 D_tb = 4'b1100;
69 @(posedge clk);
70 assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
71 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
72
73 // CHECK_3_past_state y was 01
74 D_tb = 4'b1110;
75 @(posedge clk);
76 assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=10, Got Y=%b", D_tb, Y_tb);
77 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
78
79 // CHECK_3
80 D_tb = 4'b1110;
81 @(posedge clk);
82 assert(Y_tb == 2'b10) else $error("ASSERTION FAILED: D=%b, Expected Y=10, Got Y=%b", D_tb, Y_tb);
83 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
84
85 D_tb = 4'b1010;
86 @(posedge clk);
87 assert(Y_tb == 2'b10) else $error("ASSERTION FAILED: D=%b, Expected Y=10, Got Y=%b", D_tb, Y_tb);
88 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
89
90 D_tb = 4'b0110;
91 @(posedge clk);
92 assert(Y_tb == 2'b10) else $error("ASSERTION FAILED: D=%b, Expected Y=10, Got Y=%b", D_tb, Y_tb);
93 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
94
95 D_tb = 4'b0010;

```

```

95     D_tb = 4'b0010;
96     @(posedge clk);
97     assert(Y_tb == 2'b10) else $error("ASSERTION FAILED: D=%b, Expected Y=10, Got Y=%b", D_tb, Y_tb);
98     assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
99
100    // CHECK_4_past_state y was 10
101    D_tb = 4'b0001;
102    @(posedge clk);
103    assert(Y_tb == 2'b10) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
104    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
105
106    // CHECK_4
107    D_tb = 4'b0001;
108    @(posedge clk);
109    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
110    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
111
112    D_tb = 4'b1001;
113    @(posedge clk);
114    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
115    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
116
117    D_tb = 4'b0101;
118    @(posedge clk);
119    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
120    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
121
122    D_tb = 4'b1101;
123    @(posedge clk);
124    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
125    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
126

```

```

126
127    D_tb = 4'b0011;
128    @(posedge clk);
129    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
130    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
131
132    D_tb = 4'b0111;
133    @(posedge clk);
134    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
135    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
136
137    D_tb = 4'b1011;
138    @(posedge clk);
139    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
140    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
141
142    D_tb = 4'b1111;
143    @(posedge clk);
144    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=11, Got Y=%b", D_tb, Y_tb);
145    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
146
147    // CHECK_6_past_state y was 11
148    D_tb = 4'b0100;
149    @(posedge clk);
150    assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
151    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
152
153    // CHECK_6
154    D_tb = 4'b0100;
155    @(posedge clk);
156    assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
157    assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
158

```

```
146
147 // CHECK_6_past_state y was 11
148 D_tb = 4'b0100;
149 @(posedge clk);
150 assert(Y_tb == 2'b11) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
151 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
152
153 // CHECK_6
154 D_tb = 4'b0100;
155 @(posedge clk);
156 assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
157 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
158
159 D_tb = 4'b1100;
160 @(posedge clk);
161 assert(Y_tb == 2'b01) else $error("ASSERTION FAILED: D=%b, Expected Y=01, Got Y=%b", D_tb, Y_tb);
162 assert(valid_tb == 1) else $error("ASSERTION FAILED: D=%b, Expected valid=1, Got valid=%b", D_tb, valid_tb);
163
164 $stop;
165 end
166
167 endmodule
```

```

module ALU_tb;

    reg clk;
    reg reset;
    reg [1:0] Opcode;
    reg signed [3:0] A;
    reg signed [3:0] B;
    wire signed [4:0] C;

    // Instantiate DUT
    ALU DUT (
        .clk(clk),
        .reset(reset),
        .Opcode(Opcode),
        .A(A),
        .B(B),
        .C(C)
    );

    // Clock Generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Covergroup definition
    covergroup opcode_cg @(posedge clk);
        option.per_instance = 1;
        coverpoint Opcode {
            bins add_op = {2'b00};
            bins sub_op = {2'b01};
            bins not_a_op = {2'b10};
            bins red_or_op = {2'b11};
        }
    endgroup

```

```

37 opcode_cg opcode_cov;
38
39 // stimulus
40 initial begin
41     opcode_cov = new();
42
43     reset = 1;
44     Opcode = 2'b00;
45     A = 0;
46     B = 0;
47     @(posedge clk);
48     reset = 0;
49     @(posedge clk); // Wait one more cycle after reset
50
51     // =====
52     // Test ADD Operation
53     // =====
54     Opcode = 2'b00; A = 4'sd3; B = 4'sd2;
55     @(posedge clk); // Wait for output to be registered
56     assert(C == 5'sd5) else $error("ADD FAILED: Expected 5, got %d", C);
57
58     Opcode = 2'b00; A = -4'sd3; B = 4'sd2;
59     @(posedge clk);
60     assert(C == -5'sd1) else $error("ADD FAILED: Expected -1, got %d", C);
61
62     // =====
63     // Test SUB Operation
64     // =====
65     Opcode = 2'b01; A = 4'sd5; B = 4'sd2;
66     @(posedge clk);
67     assert(C == 5'sd3) else $error("SUB FAILED: Expected 3, got %d", C);
68
69     Opcode = 2'b01; A = -4'sd3; B = 4'sd2;
70     @(posedge clk);

```

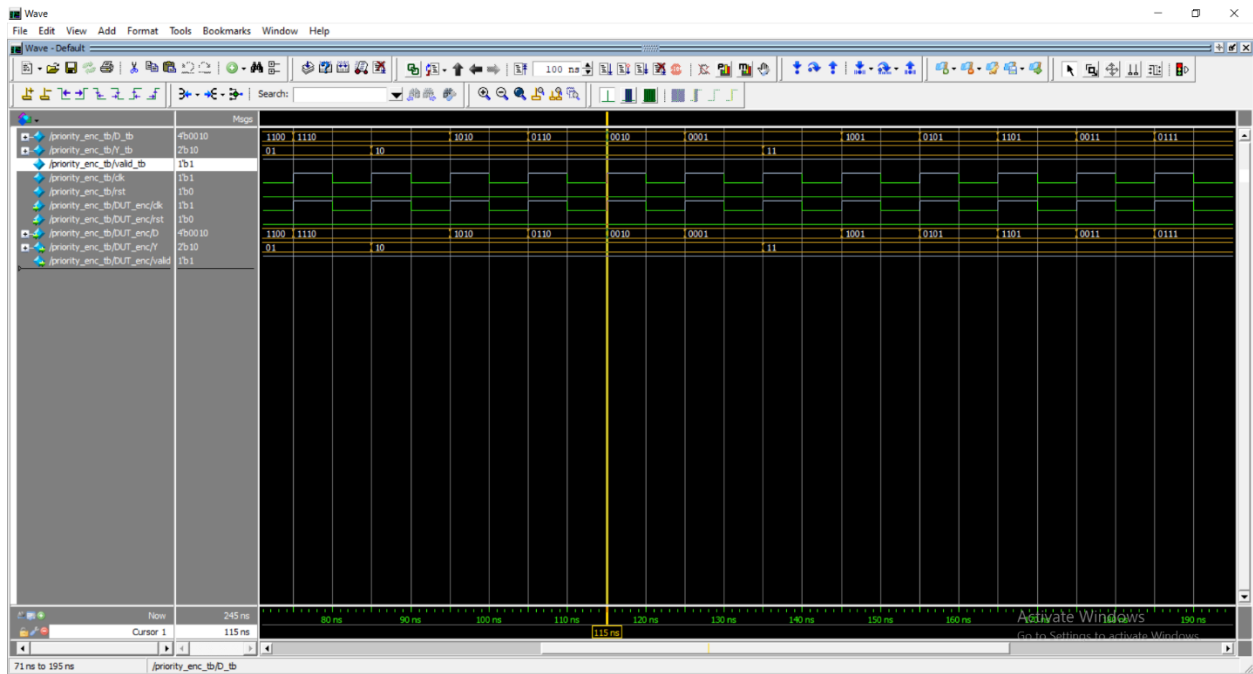


```

69 Opcode = 2'b01; A = -4'sd3; B = 4'sd2;
70 @(posedge clk);
71 assert(C == -5'sd5) else $error("SUB FAILED: Expected -5, got %d", C);
72
73 // =====
74 // Test NOT_A Operation
75 // =====
76 Opcode = 2'b10; A = 4'b1010; B = 0;
77 @(posedge clk);
78 assert(C == 5'b10101) else $error("NOT_A FAILED: Expected 10101, got %b", C);
79
80 Opcode = 2'b10; A = 4'b0101; B = 0;
81 @(posedge clk);
82 assert(C == 5'b11010) else $error("NOT_A FAILED: Expected 11010, got %b", C);
83
84 // =====
85 // Test Reduction OR Operation
86 // =====
87 Opcode = 2'b11; A = 0; B = 4'b0000;
88 @(posedge clk);
89 assert(C == 5'd0) else $error("RED_OR FAILED: Expected 0, got %d", C);
90
91 Opcode = 2'b11; A = 0; B = 4'b1010;
92 @(posedge clk);
93 assert(C == 5'd1) else $error("RED_OR FAILED: Expected 1, got %d", C);
94
95 // =====
96 // Test Reset Behavior
97 // =====
98 reset = 1;
99 @(posedge clk);
100 assert(C == 5'd0) else $error("RESET FAILED: Expected 0, got %d", C);
101 reset = 0;
102 @(posedge clk);

```

```
104 // =====
105 // Test all opcodes to ensure 100% coverage
106 // =====
107 // Test ADD with edge cases
108 Opcode = 2'b00; A = 4'sd7; B = 4'sd7;
109 @(posedge clk);
110 assert(C == 5'sd14) else $error("ADD edge case failed");
111
112 // Test SUB with edge cases
113 Opcode = 2'b01; A = -4'sd8; B = -4'sd8;
114 @(posedge clk);
115 assert(C == 5'sd0) else $error("SUB edge case failed");
116
117 // Test NOT_A with all bits
118 Opcode = 2'b10; A = 4'b1111;
119 @(posedge clk);
120 assert(C == 5'b10000) else $error("NOT_A edge case failed");
121
122 // Test RED_OR with all 1s
123 Opcode = 2'b11; B = 4'b1111;
124 @(posedge clk);
125 assert(C == 5'd1) else $error("RED_OR edge case failed");
126
127 // Wait for coverage collection
128 #20;
129 $display("All tests completed successfully!");
130 $display("Code coverage: %0f%%", $get_coverage());
131 $display("Toggle coverage: %0f%%", $get_toggle_coverage());
132 $stop;
133 end
134
135 endmodule
```



Q2

```

2  module arbiter_sva(
3      input logic clk,
4      input logic reset,
5      input logic request,
6      input logic grant,
7      input logic frame,
8      input logic irdy
9  );
10
11     // Property 1: Grant within 2-5 cycles
12     property request_to_grant;
13         @(posedge clk) disable iff (reset)
14             $rose(request) |-> ##[2:5] grant;
15     endproperty
16     assert property (request_to_grant);
17
18     // Property 2: Immediate acknowledgment
19     property grant_acknowledge;
20         @(posedge clk) disable iff (reset)
21             ($rose(grant) && $past(frame) && $past(irdy)) |->
22             (!frame && !irdy);
23     endproperty
24     assert property (grant_acknowledge);
25
26     // Property 3: Grant removal after transaction
27     property transaction_complete;
28         @(posedge clk) disable iff (reset)
29             (grant && $rose(frame) && $rose(irdy)) |=> !grant;
30     endproperty
31     assert property (transaction_complete);
32
33 endmodule

```

```

35 // Then bind to your arbiter module:
36 bind your_arbiter_module_name arbiter_sva inst (
37     .clk(clk),
38     .reset(reset),
39     .request(request),
40     .grant(grant),
41     .frame(frame),
42     .irdy(irdy)
43 );

```

Q3

```
1  // fsm_sva.sv
2  module fsm_sva(
3      input logic clk,
4      input logic reset,
5      input logic [3:0] cs,
6      input logic get_data
7  );
8
9      localparam IDLE          = 4'b0001;
10     localparam GEN_BLK_ADDR = 4'b0010;
11     localparam WAIT0        = 4'b0100;
12
13     // 1. One-hot state check
14     property state_onehot;
15         @(posedge clk) disable iff (reset)
16         |$onehot(cs);
17     endproperty
18     assert property (state_onehot);
19
20     // 2. State transition check
21     property idle_to_wait0;
22         @(posedge clk) disable iff (reset)
23         |(cs == IDLE && $rose(get_data)) | =>
24         |    (cs == GEN_BLK_ADDR) ##64 (cs == WAIT0);
25     endproperty
26     assert property (idle_to_wait0);
27
28     endmodule
```

```
29
30     // Bind to your FSM module
31     bind your_fsm_module_name fsm_sva inst (
32         .clk(clk),
33         .reset(reset),
34         .cs(cs),           // Connect to your state register
35         .get_data(get_data)
36     );
37
```